Caching with DAX – Part 2
To Begin with the Lab

Summary of the Lab

A Node.js project was created to access **Amazon DynamoDB** through **Amazon DAX** using both the low-level DynamoDB API and the high-level Document Client. The Lambda application was packaged into a ZIP file, optimized by removing the AWS SDK, and deployed to **AWS Lambda**. An IAM role with permissions for **Amazon VPC**, **Amazon DAX**, **Amazon DynamoDB**, and **Amazon CloudWatch** was configured, and the Lambda function was connected to the same VPC as the DAX cluster. Finally, both handlers were tested successfully, verifying data retrieval in DynamoDB JSON format with the low-level API and plain JSON format with the Document Client.

Understanding Amazon DAX with AWS Lambda

Amazon DAX (DynamoDB Accelerator) is an in-memory cache that sits in front of **Amazon DynamoDB** to improve the performance of read-intensive applications. It uses the same DynamoDB API, allowing applications to benefit from caching with only minor code changes. Developers can access DAX using either the low-level DynamoDB API or the high-level Document Client depending on the required response format. This reduces database read latency and improves application responsiveness.

Example:

An application stores user activity in a **UserActivity** table with **UserId** as the partition key and **Timestamp** as the sort key. When users repeatedly request the same records, **Amazon DAX** serves the cached data from memory instead of querying **Amazon DynamoDB**, resulting in much faster response times.

Lab Steps

Step 1: Create a New Node.js Project

- Create a new folder for the project (for example, **Dax_**).
- Open the folder in the terminal.
- Initialize a new Node.js project using **npm init**.
- Accept the default configuration.

```
PS C:\> cd Dax_
PS C:\Dax_> npm init -y
Wrote to C:\Dax_\package.json:

{
  "name": "dax_",
  "version": "1.0.0",
  "description": "",
  "main": "index.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "keywords": [],
  "author": "",
  "license": "ISC",
  "type": "commonjs"
}


PS C:\Dax_> dir


    Directory: C:\Dax_


Mode                 LastWriteTime         Length Name
----                 -------------         ------ ----
-a----         08-07-2026     13:28            240 package.json
```

Step 2: Install the Amazon DAX Client

- Install the **Amazon DAX Client** package using NPM.
- Save the dependency in the project.
- **npm install amazon-dax-client –save**

```
PS C:\Dax_> npm install amazon-dax-client aws-sdk
npm warn deprecated querystring@0.2.0: The querystring API is considered Legacy. new code should use the URLSearchParams API instead.
npm warn deprecated uuid@3.4.0: uuid@10 and below is no longer supported.  For ESM codebases, update to uuid@latest.  For CommonJS codebases, use uuid@11 (b
ut be aware this version will likely be deprecated in 2028).
npm warn deprecated uuid@8.0.0: uuid@10 and below is no longer supported.  For ESM codebases, update to uuid@latest.  For CommonJS codebases, use uuid@11 (b
ut be aware this version will likely be deprecated in 2028).
npm warn deprecated aws-sdk@2.1693.0: The AWS SDK for JavaScript (v2) has reached end-of-support, and no longer receives updates. Please migrate your code t
o use AWS SDK for JavaScript (v3). More info https://a.co/cUPnyil

added 50 packages, and audited 51 packages in 6s

19 packages are looking for funding
  run `npm fund` for details

3 moderate severity vulnerabilities

Some issues need review, and may require choosing
a different dependency.

Run `npm audit` for details.
npm warn allow-scripts 1 package has install scripts not yet covered by allowScripts:
npm warn allow-scripts   aws-sdk@2.1693.0 (postinstall: node scripts/warn-maintenance-mode.js)
npm warn allow-scripts
npm warn allow-scripts Run `npm approve-scripts --allow-scripts-pending` to review, or `npm approve-scripts <pkg>` to allow.
PS C:\Dax_> dir


    Directory: C:\Dax_


Mode                 LastWriteTime         Length Name
----                 -------------         ------ ----
d-----         08-07-2026     13:37                node_modules
-a----         08-07-2026     13:37          23316 package-lock.json
-a----         08-07-2026     13:37            328 package.json
```

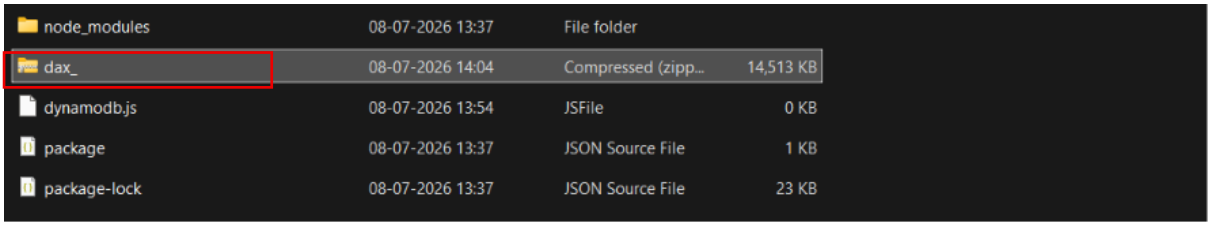
Step 3: Create the Low-Level DynamoDB Lambda Handler

- Create a file named **DynamoDB.js**.
- Import the **AWS SDK** and **Amazon DAX Client**.

Step 4: Package the Lambda Project

- Select all project files.
- Create a ZIP archive.
- Open the ZIP file.
- Navigate to the **node_modules** folder.

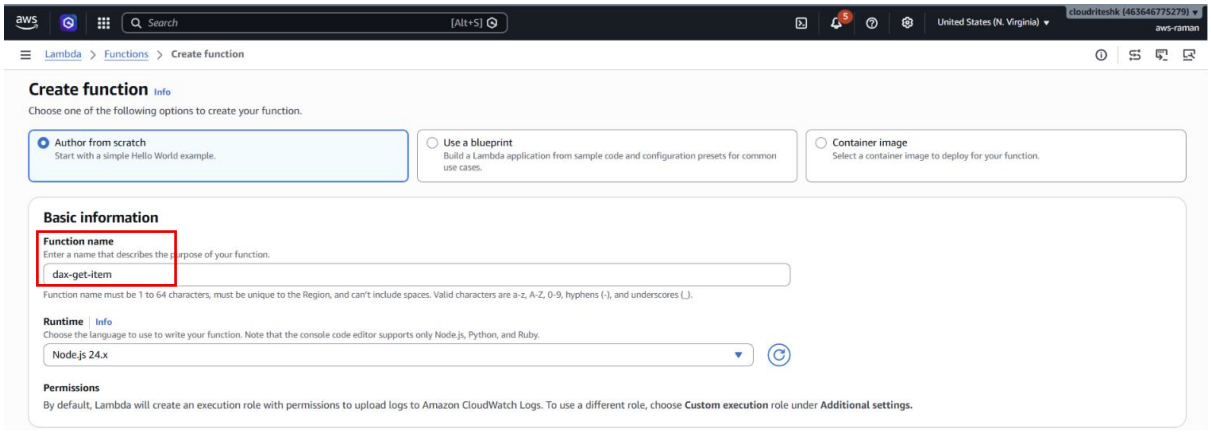
- Delete the **aws-sdk** folder because the AWS SDK is already available in the **AWS Lambda** runtime.
- Save the ZIP file.



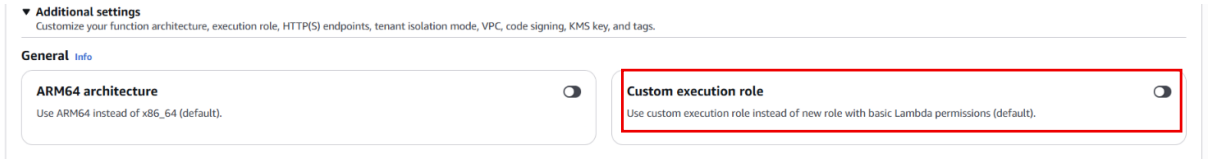
Step 5: Create the Lambda Function

- Sign in to the **AWS Management Console**.
- Navigate to **AWS Lambda**.
- Click **Create function**.

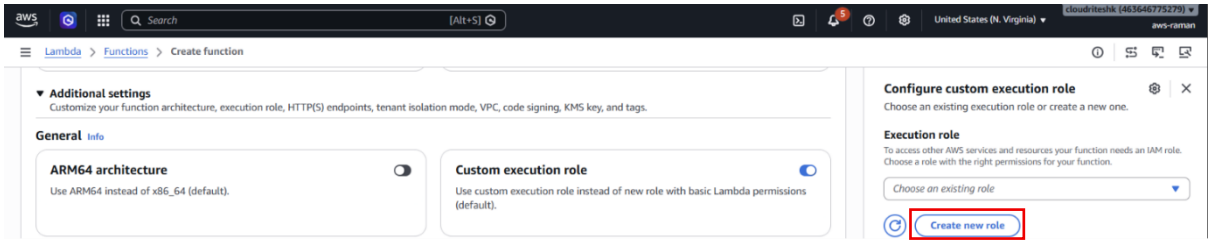
- Enter the function name **dax-get-item**.



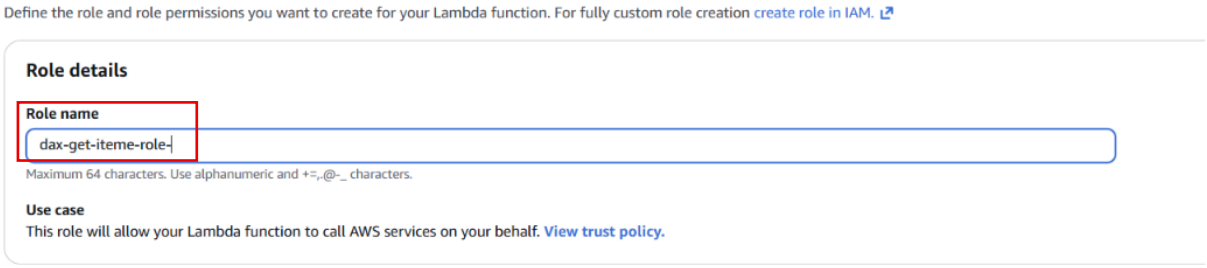
- Go to Additional setting.
- Navigate to **Custom execution role**



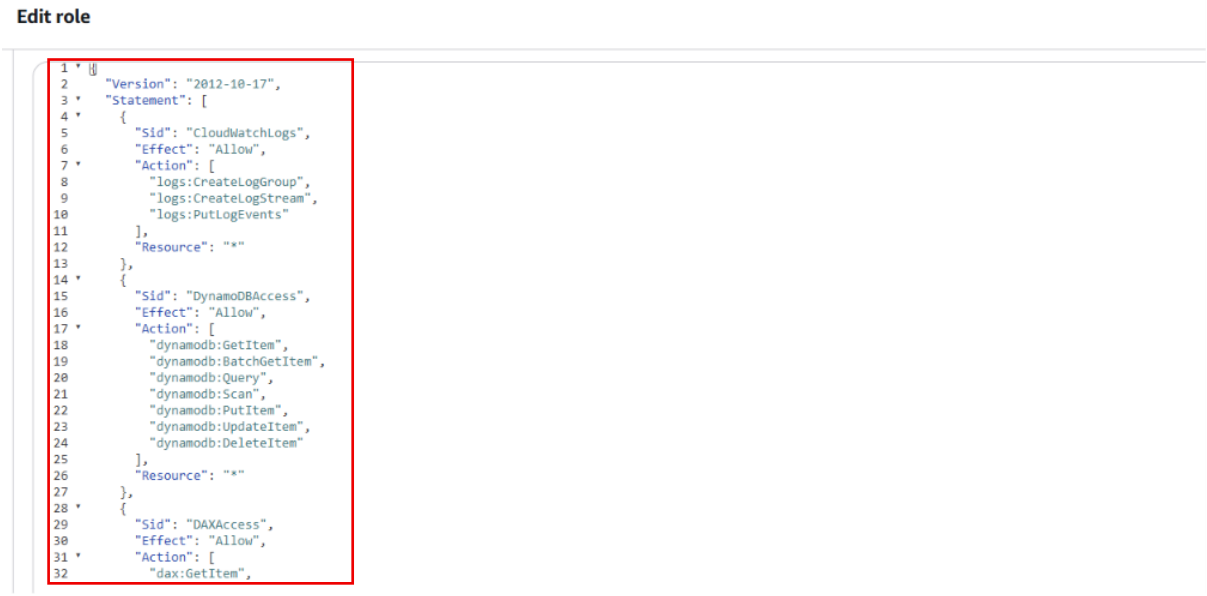
- Click on the **Create new role**



- Create a new execution role named dax-get-iteme-role-



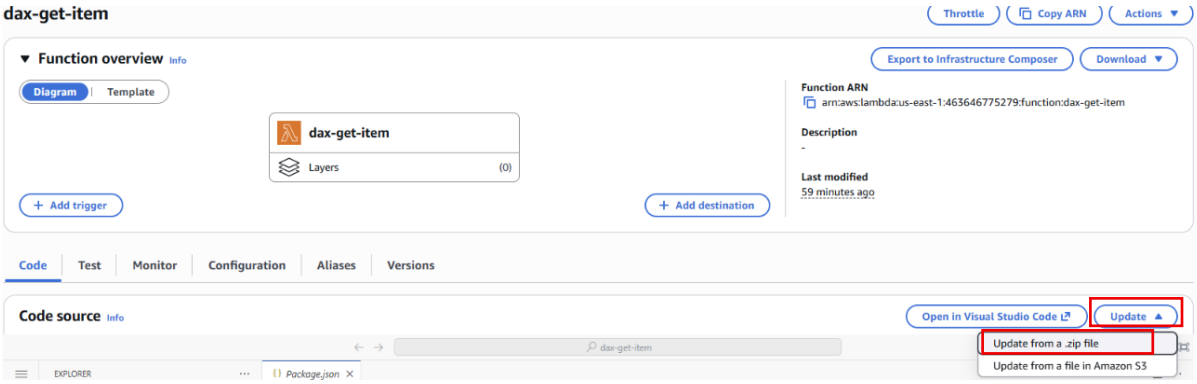
- Open the IAM policy document.
- Click **Edit**.
- Paste the IAM policy JSON that grants access to **Amazon VPC**, **Amazon DAX**,
- **Amazon DynamoDB**, and **Amazon CloudWatch**.



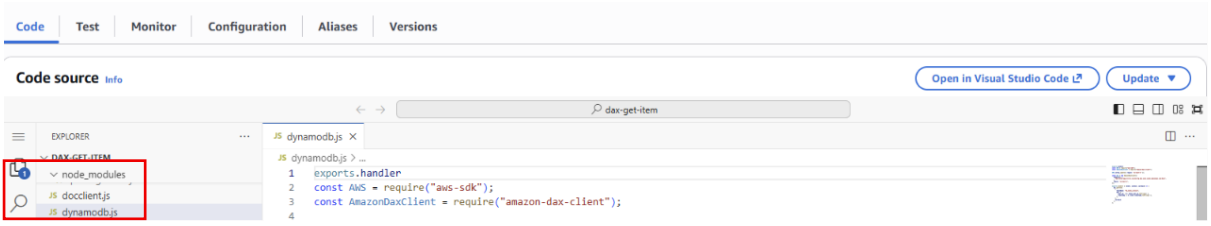
- Click **Allow**.
- Create the Lambda function.

Step 6: Upload the Lambda Code

- Under **Function code**, choose **Upload from → .zip file**.



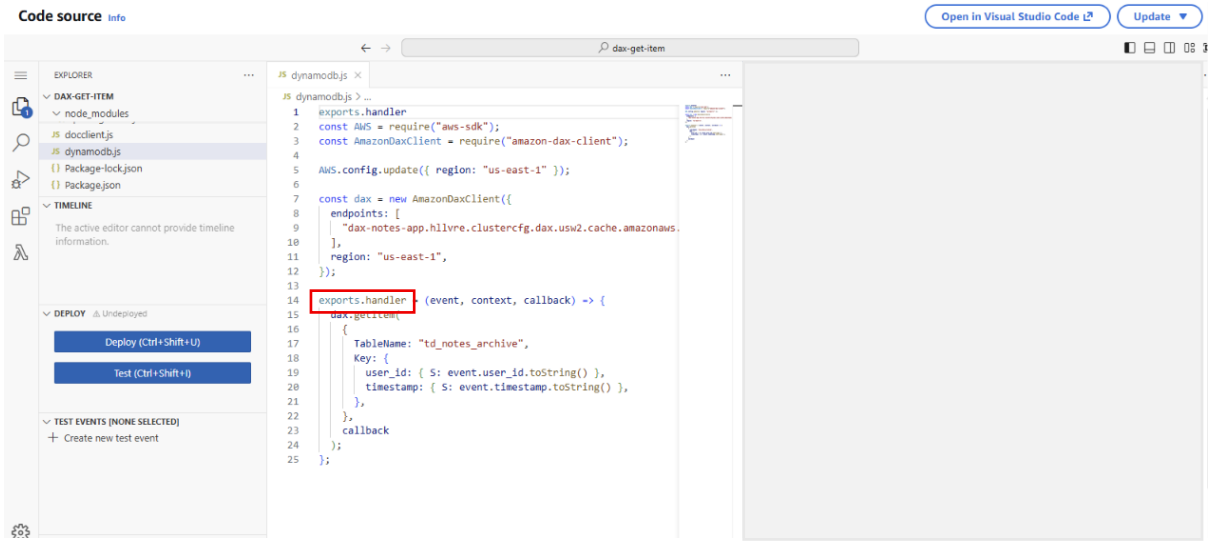
- Select the ZIP archive created earlier.
- ZIP file updated successfully



- If the upload fails, upload the ZIP file to **Amazon S3** and choose **Upload from Amazon S3**.

Step 7: Configure the Low-Level DynamoDB Handler

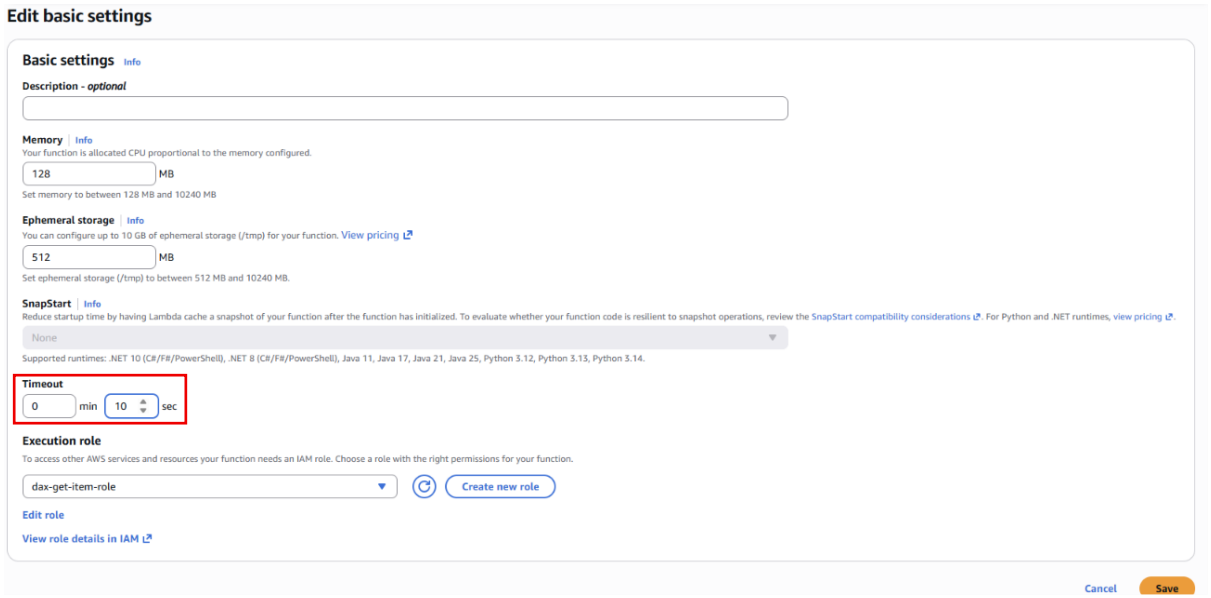
- Set the handler name to:**DynamoDB.handler**



- Click on the Deploy

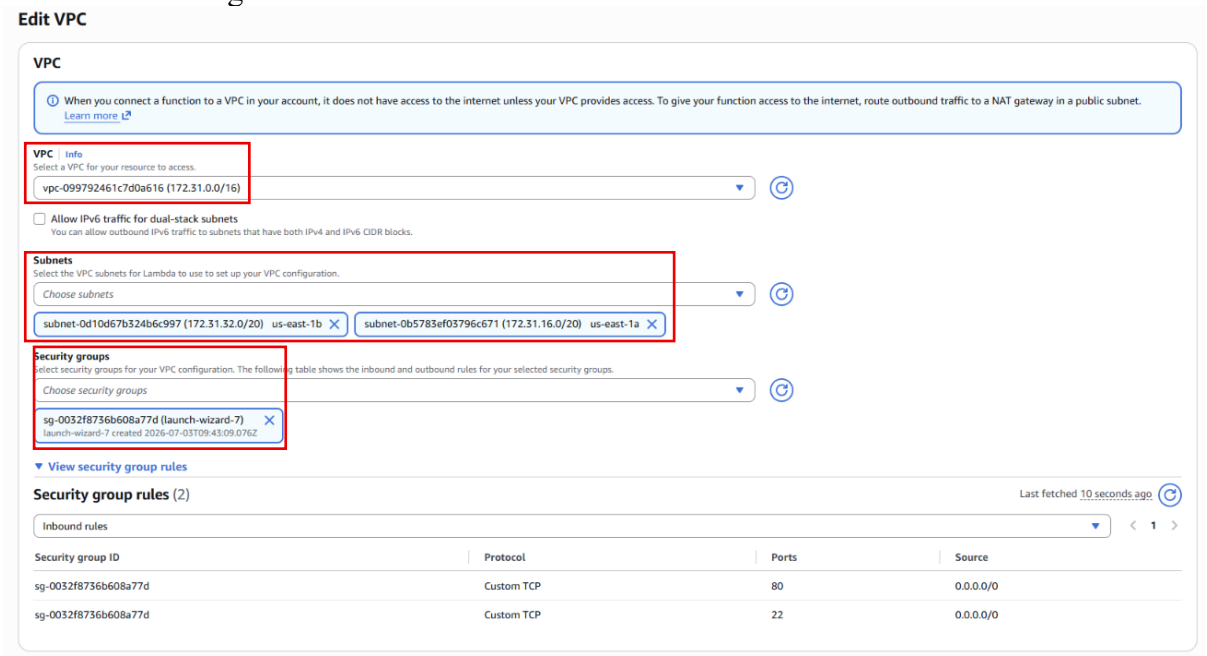
Step 8: Configure the Lambda Network

- Go to Configuration → General configuration.
- Increase the Lambda timeout to **10 seconds**.



- Open the **Network** settings.
- Select the same **Amazon VPC** used by the DAX cluster.
- Select the required private subnets.
- Choose the corresponding security group.

- Save the changes.

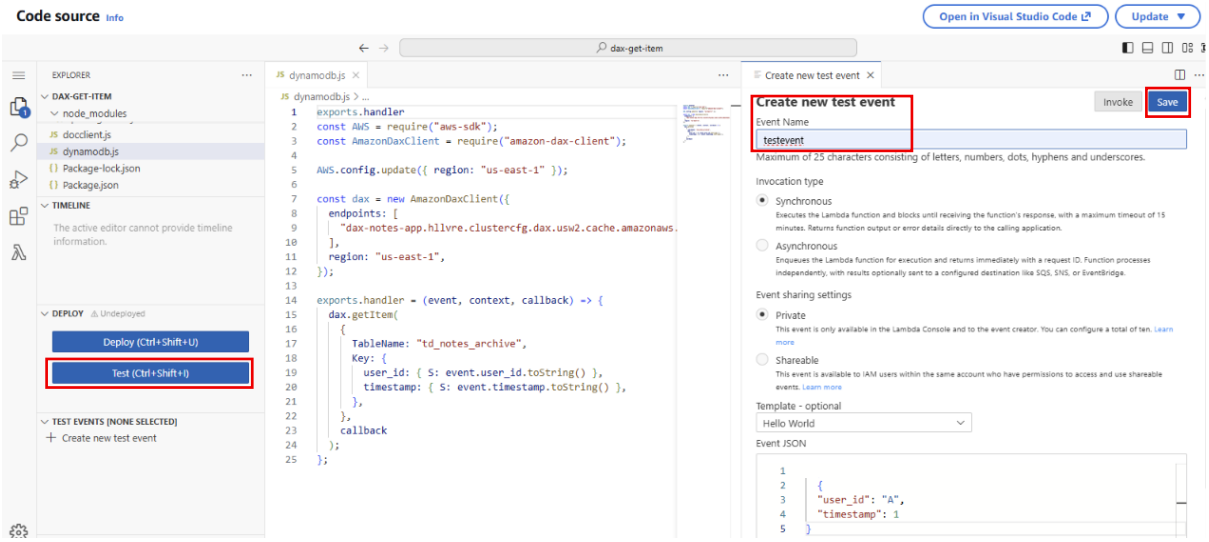


Step 9: Test the Low-Level DynamoDB Handler

- Click **Test**.
- Create a new test event named **testevent**.
- Provide the following event:

```
{
 "user_id": "A",
 "timestamp": "1"
}
```

- Save the test event.



- Run the test.
- Verify that the item is returned successfully in **DynamoDB JSON** format.

Step 10: Test the Document Client Handler

- Change the Lambda handler to:

- Save the function.
- Run the same test event again.
- Verify that the response is returned successfully in standard JSON format.
- Confirm that both the low-level API and the **Amazon DynamoDB Document Client** retrieve data through **Amazon DAX** successfully.